

```
0000 -----
compiler-- Copyright (C) 2024-2026 Vincent MORIN - Universit  de Bretagne Occidentale-- This program is free software: you can redistribute it and/
or modify-- it under the terms of the GNU General Public License as published by-- the Free Software Foundation, either version 3 of the License, or
-- (at your option) any later version.-- This program is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without eve
n the implied warranty of-- MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the-- GNU General Public License for more details.-- You sh
ould have received a copy of the GNU General Public License-- along with this program. If not, see <https://www.gnu.org/licenses/>.--
-----
--IDL.ADS-VINCENT MORIN-21/6/2024--UNIVERSITE DE BRETAGNE OCCIDENTALE-(UBO)
-----
DIANA_NODE_ATTR_CLASS_NAMES;package IDL is
PROJECT_PATH: STRING(1..256) := (others => ' ');
CHEMIN D ACCES AU REPERTOIRE PROJ T-QUI CONTIENT LA LIBRAIRIE /ADA_LIB PROJECT_PATH_LENGTH: NATURAL := 0;
LONGUEUR DU CHEMIN PROJ T LIB_PATH: STRING(1..256) := (others => ' ');
CHEMIN D ACCES AU REPERTOIRE LIBRAIRIE LIB_PATH_LENGTH: NATURAL := 0;
DEFAULT_LIB_PATH: STRING(1..11) := "/ADA_LIB/";
CHEMIN D ACCES AU REPERTOIRE LIBRAIRIE C_R-E A T I O-N / O U-V E R T U-R E F E-R M E T U-R E D E- F I C H-I E R
D-I A N A A N A procedure CREATE_IDL_TREE_FILE( PAGE_FILE_NAME : STRING );
CREE UN FICHIER-D'ARBRE DIANA procedure OPEN_IDL_TREE_FILE( PAGE_FILE_NAME : STRING );
OUVRE UN FICHIER D'ARBRE D JA EXISTANT procedure CLOSE_IDL_TREE_FILE;
FERMETURE DU FICHIER D'ARBRE procedure DELETE_IDL_TREE_FILE;
E T-A P E S-F R O N T-A L E S-D E C O-M P I L A-T I O N procedure PAR_PHASE( PATH_TEXTE, NOM_TEXTE, LIB_PATH : STRING );
ANALYSE SYNTAXIQUE DU FICHIER "NOM_TEXTE", PRODUIT UN FICHIER D'ARBRE PARTIELLEMENT ATTRIBUE procedure LIB_PHASE;
INSERTION DES ELEMENTS DE-LIBRAIRIE-UTILISES DANS L'ARBRE DIANA procedure SEM_PHASE;
ANALYSE SEMANTIQUE DU FICHIER ARBRE-DIANA procedure ERR_PHASE( ACCES_TEXTE : STRING );
ECRITURE DES ERREURS POUR-LES ANALYSES SUR L'ARBRE procedure WRITE_LIB;
MODIFICATION DES INFORMATIONS DE LIBRAIRIE SUITE A LA COMILATION DE "NOM_TEXTE.DIA" procedure PRETTY_DIANA( OPTION : CHARACTER := 'U' );
DISPOSITIF D'ACCES A UN-ARBRE DIANA. POUR UTILISER LES ELEMENTS-ATTRIBUTE_NAME ET NODE_NAME ET LES CLASS,, -- IL FAUT SE REFERER AU DOCUMENT DE-L'IDL DIANA
type SHORT is range -32_768 .. 32767;
for SHORT'SIZE use 16;
type POSITIVE_SHORT is range 0 .. 32767;
for POSITIVE_SHORT'SIZE use 15;
type PAGE_IDX is range 0 .. 16#7FFF;
for PAGE_IDX'SIZE use 15;
type LINE_IDX is range 0 .. 128;
type SRCOL_IDX is range 0 .. 255;
for SRCOL_IDX'SIZE use 8;
type VPTR_TYPE is ( P, S, L, H I );
PTR NOEUD, SOURCE_POS, LIST, HEADER/INTEGER type TREE ( PT : VPTR_TYPE := P ) is record
PAR DEFAUT POINTEUR DE NOEUD c
ase PT is
when P | L => POINTEUR NORMAL-DE NOEUD OU ATTRIBUT LISTE
TY: NODE_NAME;
TYPE DE NOEUD
PG: PAGE_I
DX;
REFERENCE DE PAGE VIRTUELLE
LN: LINE_IDX;
DECALAGE DANS UNE PAGE VIRTUELLE
when S => POINTEUR DE SOURCE
LINE AVEC COLONNE SOURCE EN PLACE DU TYPE
COL: SRCOL_IDX;
NUMERO DE COLONNE DANS LE-TEXTE SOURCE
SPG: PAGE_IDX;
REFERENCE DE PAGE VIRTUELLE
SLN: LINE_IDX;
DECALAGE DANS UNE PAGE VIRTUELLE
when HI => HEADER DE NOEUD-OU INTEGER ( SH
ORT) CODE PAR VALEUR ABSOLUE ET INDICATEUR DE-COMPLEMENT A DEUX
NOTY: NODE_NAME;
TYPE DE NOEUD
ABSS: POSITIVE_SHORT;
VALEUR ABSOLUE D UN SHORT-POUR UNE VALEUR ENTIERE
NSIZ: ATTR_NBR;
NOMBRE D ATTRIBUTS DU NOEUD (SI ENTETE) OU INDICATEUR DE COMPLE
MENT 0+ 1- POUR ABSS
end case;
record;
TREE'SIZE use 32;
TREE use record-at mod 4;
PT-at 0 range 0..1;
LN-at 0 range 2..8;
SLN-at 0 range 2..8;
NSIZ-at 0 range 2..8;
PG-at 0 range 9..23;
SPG-at 0 range 9..23;
ABSS-at 0 range 9..23;
COL-at 0 range 24..31;
TY-at 0 range 24..31;
NOTY-at 0 range 24..31;
end record;
TREE NIL: constant-TREE := ( P, TY=> DN_NIL, PG => 0, LN => 0 );
TREE VOID: constant-TREE := ( P, TY=> DN_VOID, PG => 0, LN => 0 );
POINTEUR NON TYPE MAIS PAS NIL
EN GENERAL (POINTE QUELQUE CHOSE)
TREE_ROOT: constant-TREE := ( P, TY=> DN_ROOT, PG => 1, LN => 0 );
type SEQ_TYPE is record
FIRST, NEXT: TREE;
end record;
A C-C E S A L ' A R-B R E function MAKE( NN : NODE_NAME ) return TREE;
AJOUTE UN NOEUD-DE TYPE NODE_NAME
procedure D( AN : ATTRIBUTE_NAME; T : TREE; V : TREE );
ECRITURE D'UN ATTRIBUT CONTENANT UN-ARBRE function DB( AN : ATTRIBUTE_NAME; T : TREE; V : BOOLEAN );
ECRITURE D'UN ATTRIBUT CONTENANT UN-BOOLEEN
function DB( AN : ATTRIBUTE_NAME; T : TREE; V : BOOLEAN );
LECTURE D'UN ATTRIBUT CONTENANT UN BOOLEEN
procedure DI( AN : ATTRIBUTE_NAME; T : TREE; V : INTEGER );
ECRITURE D'UN ATTRIBUT CONTENANT UN-ENTIER (16 BITS)
function DI( AN : ATTRIBUTE_NAME; T : TREE; V : INTEGER );
LECTURE D'UN ATTRIBUT CONTENANT UN ENTIER (16-BITS)
function LIST( T : TREE ) return SEQ_TYPE;
REND LA LISTE CONTENUE DANS UN NOEUD POINTE PAR LE POINTEUR D'ARBRE
function IS_EMPTY( S : SEQ_TYPE ) return BOOLEAN;
TESTE-UNE LISTE
procedure POP( S : in out SEQ_TYPE; T : out TREE );
EXTRAIT UN ELEMENT DE LISTE ET REPOINTE S SUR-LE RESTE
function PRINT_NAME( T : TREE ) return STRING;
TXTREP OR SYMBOL_REP
function PRINT_NUM( T : TREE ) return STRING;
DN_NUM_VAL
function NODE_IMAGE( NN : NODE_NAME ) return STRING;
CHAINE REPRESENTANT UN NOEUD
function ATTR_IMAGE( AN : ATTRIBUTE_NAME ) return STRING;
CHAINE REPRESENTANT U
N NOM D'ATTRIBUT
function GET_LIB_PREFIX return STRING;
UTILISE PAR LIB_PHASE ET-WRITE_LIB
function ANONYMOUS_NAME_AT( T : TREE ) return STRING;
UTILISE PAR EXPANDER
procedure DEBUG_STOP;
package PRINT_NOD
AFFICHAGE DE TOUT OU PARTIE DE L'A
RBRE
is
procedure PRINT_TREE( T : TREE );
function L_PRINT_TREE( T : TREE ) return NATURAL;
RETOURNE LE NOMBRE DE CARACTERES
procedure PRINT_NODE( T : TREE; INDENT : NATURAL := 0 );
end-PRINT_NOD;
pragma INLINE ( DB );
pragma INLINE ( DI );
pragma INLIN
0050 E ( D );
end-IDL;
end-1-2-3-4-5-6-7-8-9-0-1-2
```